

Implementación y análisis de complejidad de listas, pilas y colas en Java

Hayran Andrés López González

Objetivo del taller

Implementar las estructuras de datos List (listas enlazadas), Stack (pila) y Queue (cola) en Java, abarcando tanto arreglos dinámicos como listas enlazadas, realizando un análisis de complejidad de los métodos asociados.

Explicación de la implementación

Este trabajo ha sido implementado en el siguiente repo de GitHub:

https://github.com/Andresit1524-Uni/lab1_edd

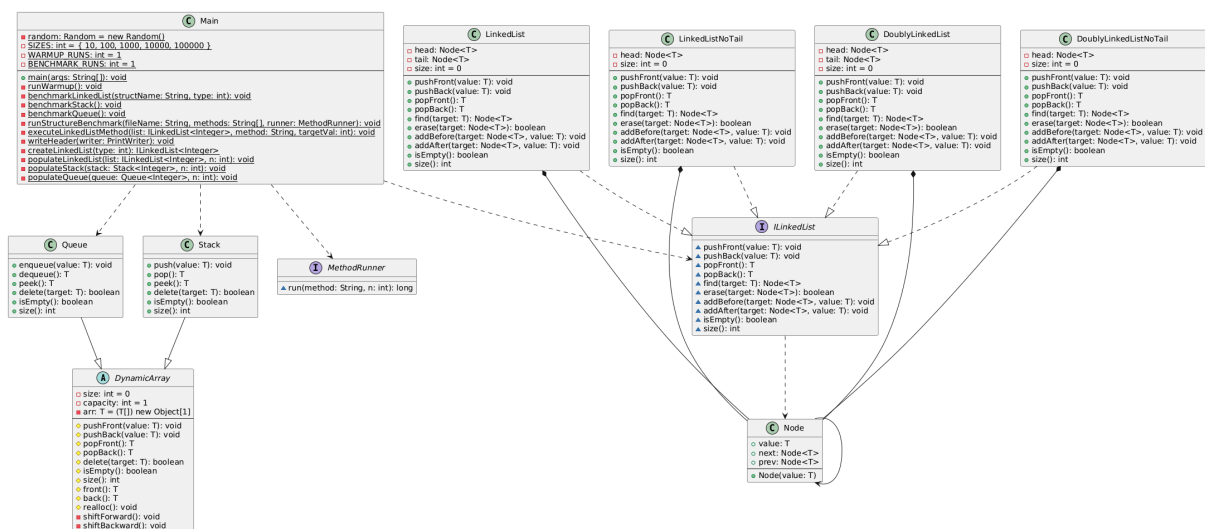
La implementación se rige por la siguiente estructura:

- Una interfaz `ILinkedList` con los métodos comunes para todas las listas enlazadas (insertar, eliminar, buscar, etc.).
- Una clase abstracta `DynamicArray` con los métodos de un arreglo dinámico ya implementados. Es abstracta porque no la usaremos directamente.

De aquí parten las clases que si se usan:

- Las 4 variantes de listas enlazadas, todas implementando la interfaz `ILinkedList`:
 - `LinkedList`
 - `LinkedListNoTail`
 - `DoublyLinkedList`
 - `DoublyLinkedListNoTail`
- `Stack` y `Queue` para las pilas y colas respectivamente, basados en la clase abstracta `DynamicArray`

El diagrama UML de este proyecto es el siguiente:



Análisis de complejidad

Listas enlazadas

Simple

- Buscar siempre toma $O(n)$, y es el cuello de botella de los siguientes métodos
- Insertar y eliminar al frente es inmediato porque tenemos un puntero a la cabeza de la lista y el siguiente nodo. Complejidad $O(1)$
- Insertar y eliminar al final requiere buscar el último nodo y el penúltimo (al eliminar), por lo que la complejidad es $O(n)$
- Añadir antes de un nodo requiere buscar el anterior ($O(n)$) pero añadir después se puede hacer de inmediato porque tenemos acceso al siguiente nodo ($O(1)$)
- Eliminar un nodo requiere buscar el anterior, entonces su complejidad es $O(n)$
- Verificar que está vacío es tan sencillo como ver si hay cabeza. Complejidad $O(1)$

Ten en cuenta que los métodos `addBefore` y `addAfter` ofrecen un puntero directo al nodo, entonces por ello pueden ser más rápidos.

Simple (con cola)

El nuevo puntero con cola hace que añadir elementos al final sea más sencillo porque tenemos la cola directamente. Pero el resto de operaciones siguen igual en cuanto requieren búsquedas (especialmente para buscar el nodo anterior a uno dado).

Dobles

Las listas enlazadas dobles introducen un puntero anterior que nos permite trabajar hacia atrás también. Gracias a eso, ahora borrar nodos y añadir antes de uno puede hacerse en tiempo constante porque podemos ver hacia atrás y cablear directamente.

Pero como no tienen puntero a cola, aún debemos buscarla, degradando las operaciones al final de la lista.

Dobles (con cola)

Ahora con el puntero a cola, podemos ir directo al final de la lista y operar sobre ella, haciendo que la adición/eliminación de elementos se torne de tiempo constante.

Con estas consideraciones, la tabla de complejidades queda como se ve a continuación. En azul las operaciones con rendimiento mejorado y en rojo las que quedan peor a comparación.

Operación	Simple	Simple (con cola)	Doble	Doble (con cola)
<code>pushFront</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>pushBack</code>	$O(n)$	$O(1)$	$O(n)$	$O(1)$
<code>popFront</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>popBack</code>	$O(n)$	$O(n)$	$O(n)$	$O(1)$
<code>find</code>	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<code>erase</code>	$O(n)$	$O(n)$	$O(1)$	$O(1)$
<code>addBefore</code>	$O(n)$	$O(n)$	$O(1)$	$O(1)$
<code>addAfter</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>isEmpty</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$

Comparando las tablas obtenemos que la mejor estructura en términos de rendimiento general es la **lista enlazada doble con puntero a cola**, al permitir un avance en dos direcciones y acceso a los dos extremos.

Arreglos dinámicos, pilas y colas

Los arreglos dinámicos son arreglos cuyo tamaño puede crecer de manera indefinida, permitiendo acceder a elementos de manera aleatoria (por índice) y reasignarse automáticamente cuando se llena.

Eliminar o añadir un elemento de una posición diferente a la última requiere trasladar el resto de elementos hacia adelante o hacia atrás, lo cual hace que su complejidad sea de $O(1)$. El resto de operaciones pueden hacerse de manera inmediata y por ende con $O(1)$.

La complejidad de sus operaciones es la siguiente:

Operación	Pila	Cola	Complejidad
pushFront	-	-	$O(n)$
pushBack	push	enqueue	$O(1)$
popFront	-	dequeue	$O(n)$
popBack	pop	-	$O(1)$
delete	delete	delete	$O(n)$
isEmpty	isEmpty	isEmpty	$O(1)$
size	size	size	$O(1)$
front	-	front	$O(1)$
back	peek	-	$O(1)$

Ten en cuenta que se están usando arreglos dinámicos llenados de inicio a fin. Con arreglos circulares se puede lograr más rendimiento, y con arreglos circulares dinámicos (no implementados en este caso) también una capacidad máxima arbitraria.

Demostración de que la inserción es $O(1)$

Como se vió en clase, insertar elementos al final de un arreglo toma un tiempo constante. Tomaré mi versión favorita de la demostración: el costo amortizado. El costo amortizado c de n operaciones es el costo promedio:

$$c = \frac{C}{n}$$

El costo individual de añadir un elemento al arreglo depende de si hay capacidad o si debemos reagisnar:

$$c_i = 1 + \begin{cases} i - 1 & \text{si } i - 1 \text{ es potencia de } 2 \\ 0 & \text{en otro caso} \end{cases}$$

Entonces el costo total C es la suma de los costos individuales:

$$\begin{aligned} C &= \sum_{i=1}^n c_i = n + \sum_{i=1}^{\lfloor \log_2(n-1) \rfloor} 2^i \\ &= n + 2^{\lfloor \log_2(n-1) \rfloor + 1} - 1 \end{aligned}$$

Y puesto que $2^{\lfloor \log_2(n-1) \rfloor + 1} \leq 2(n-1) < 2n$ entonces obtenemos la desigualdad:

$$C < n + 2n = 3n$$

Volviendo al costo amortizado obtenemos:

$$c = \frac{C}{n} = \frac{3n}{n} = 3$$

El costo está acotado a 3, lo cual es **constante**.

Gráficas y resultados empíricos

Para analizar el comportamiento real de estos métodos utilizaremos medidas basadas en `System.nanoTime()`. El esquema de medición es el siguiente:

- Mediremos el rendimiento con 10, 100, 1000, ..., hasta 10^5 operaciones
- Esta medición es por cada método de cada clase implementada, en un promedio de varios intentos (BENCHMARK_RUNS)
- Haremos la tabla de tiempos y gráficas para los mismos. Una tabla por cada estructura de datos
- Las gráficas serán en escala logarítmica para evidenciar todas las escalas adecuadamente. Una gráfica por cada tabla
- Se usarán **nanosegundos** para las mediciones

Funcionamiento de la Suite:

1. **Calentamiento:** Ejecuta pasadas iniciales descartadas para permitir que la JVM realice optimizaciones JIT.
2. **Escala exponencial:** Evalúa cada método con tamaños $n \in \{10, 100, 1000, 10000, 100000\}$.
3. **Población con datos aleatorios:** Se llenan las estructuras previamente usando valores aleatorios para métodos de eliminación o consulta.
4. **Archivos CSV matriciales separados:** Se genera un archivo .csv independiente por cada estructura de datos.

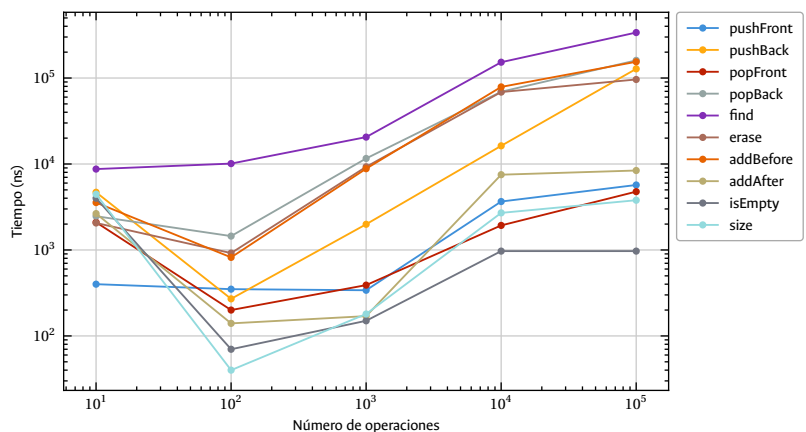
El código para registrar estos tiempos no interfiere con el rendimiento de los algoritmos, y se ubica toda en la clase `Main`. Las gráficas se crean con **lilaq**, una librería de **Typst**, el mismo sistema con el que se ha compilado este PDF. los datos estarán también registrados en un CSV en `data/`.

Listas enlazadas

1. Simples

Los métodos `pushBack`, `popBack`, `find`, `erase` y `addBefore` poseen una complejidad lineal, mientras que el resto son constantes. La falta de puntero de cola hace de las suyas.

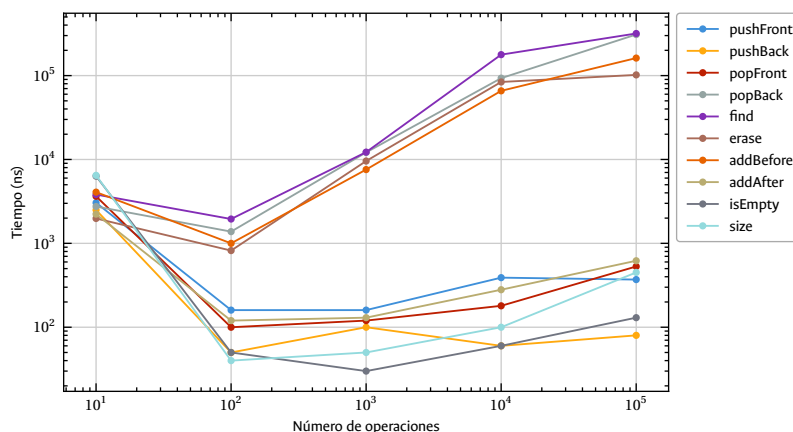
Método	10	100	1000	10000	100000
pushFront	400	350	340	3660	5700
pushBack	4690	270	1990	16270	127890
popFront	2100	200	390	1930	4770
popBack	2470	1450	11600	69220	160460
find	8730	10110	20550	152770	338140
erase	2060	920	9270	68720	96210
addBefore	3560	820	8820	78840	154500
addAfter	2650	140	170	7510	8400
isEmpty	4000	70	150	970	970
size	4450	40	180	2700	3790



2. Simples (con cola)

Aquí pushBack se desploma porque el puntero de cola nos permite añadir al instante. El resto de métodos siguen lineales, especialmente popBack que nos sigue exigiendo una búsqueda.

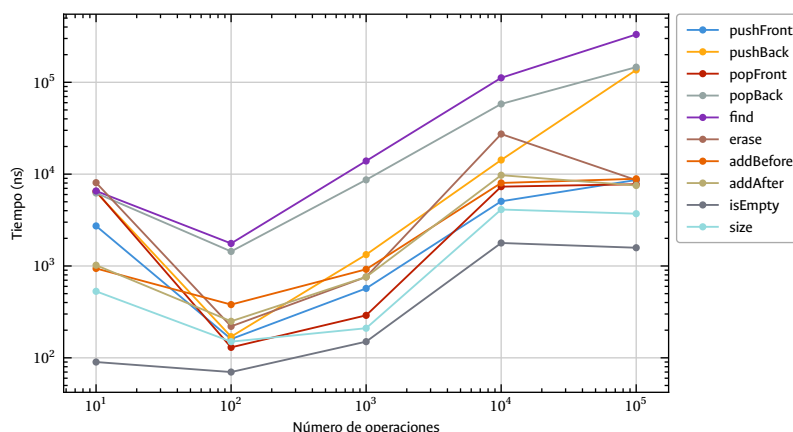
Método	10	100	1000	10000	100000
pushFront	3050	160	160	390	370
pushBack	2500	50	100	60	80
popFront	3640	100	120	180	530
popBack	2750	1380	12140	93180	308860
find	3820	1950	12230	177740	317660
erase	1980	820	9580	83950	101930
addBefore	4080	1000	7580	65810	161770
addAfter	2230	120	130	280	620
isEmpty	6330	50	30	60	130
size	6470	40	50	100	450



3. Dobles

Las listas dobles desploman erase y addBefore porque el recorrido hacia atrás nos ahorra toda la búsqueda. Pero la falta de cola nos obliga a buscar hacia el final por lo que popBack vuelve a ser lineal.

Método	10	100	1000	10000	100000
pushFront	2730	160	570	5060	8640
pushBack	6390	170	1330	14260	136370
popFront	6490	130	290	7310	7760
popBack	6230	1440	8670	58110	146270
find	6560	1760	13910	111880	331990
erase	8080	220	760	27340	8620
addBefore	940	380	920	8020	8900
addAfter	1020	250	760	9740	7520
isEmpty	90	70	150	1780	1580
size	530	150	210	4120	3710

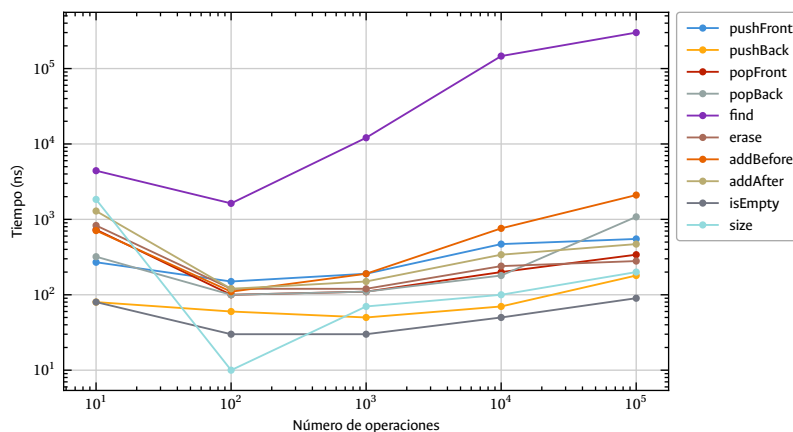


4. Dobles (con cola)

Con los dobles enlaces y el puntero a cola, ahora popBack se ejecuta en tiempo constante. El resto de métodos también se vuelven constantes por las razones explicadas arriba.

Es la mejor estructura en rendimiento, exigiendo solo cuando tenemos que buscar un elemento (find).

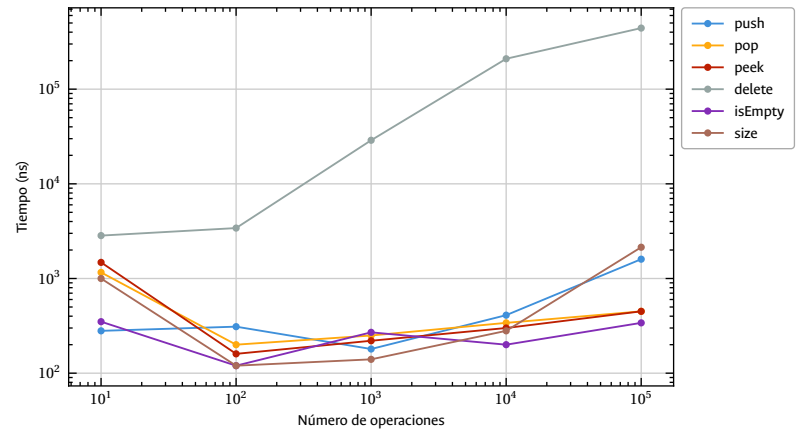
Método	10	100	1000	10000	100000
pushFront	270	150	190	470	550
pushBack	80	60	50	70	180
popFront	730	100	110	200	340
popBack	320	100	110	180	1080
find	4420	1630	12090	146430	299960
erase	830	120	120	240	280
addBefore	710	110	190	760	2100
addAfter	1290	120	150	340	470
isEmpty	80	30	30	50	90
size	1840	10	70	100	200



Pilas

Las pilas exhiben un excelente rendimiento para todas las operaciones... excepto eliminar. Por suerte esta operación **¡no es parte de las pilas!**

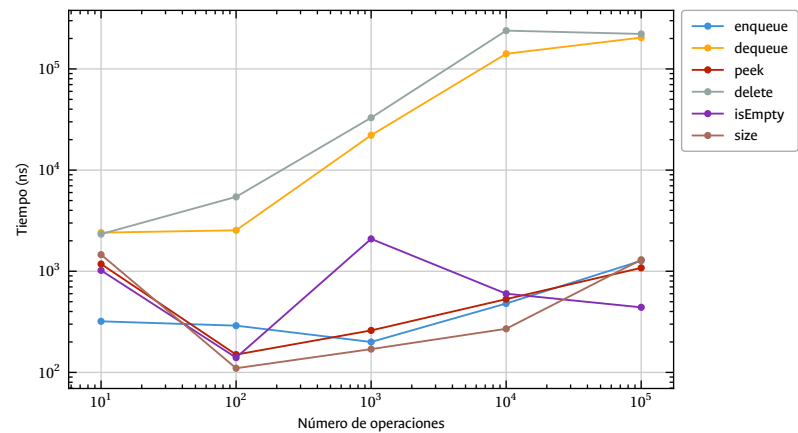
Método	10	100	1000	10000	100000
push	280	310	180	410	1600
pop	1160	200	250	340	450
peek	1480	160	220	300	450
delete	2840	3410	28850	209460	441760
isEmpty	350	120	270	200	340
size	1000	120	140	280	2140



Colas

Las colas pierden rendimiento en dequeue porque implica sacar un elemento del principio y trasladar los demás. Eliminar igualmente es ineficiente, pero también está fuera de las colas.

Método	10	100	1000	10000	100000
enqueue	320	290	200	480	1270
dequeue	2410	2540	22110	141210	204040
peek	1180	150	260	530	1080
delete	2330	5440	32930	239120	221730
isEmpty	1020	140	2090	600	440
size	1460	110	170	270	1300



Discusión

Complejidades obtenidas

Estas son las complejidades de los métodos tras el análisis empírico.

Listas enlazadas

Operación	Simple	Simple (con cola)	Doble	Doble (con cola)
pushFront	$O(1)$	$O(1)$	$O(1)$	$O(1)$
pushBack	$O(n)$	$O(1)$	$O(n)$	$O(1)$
popFront	$O(1)$	$O(1)$	$O(1)$	$O(1)$
popBack	$O(n)$	$O(n)$	$O(n)$	$O(1)$
find	$O(n)$	$O(n)$	$O(n)$	$O(n)$
erase	$O(n)$	$O(n)$	$O(1)$	$O(1)$
addBefore	$O(n)$	$O(n)$	$O(1)$	$O(1)$
addAfter	$O(1)$	$O(1)$	$O(1)$	$O(1)$
isEmpty	$O(1)$	$O(1)$	$O(1)$	$O(1)$

Pilas y colas

Operación	Pila	Cola	Complejidad
pushFront	-	-	$O(n)$
pushBack	push	enqueue	$O(1)$
popFront	-	dequeue	$O(n)$
popBack	pop	-	$O(1)$
delete	delete	delete	$O(n)$
isEmpty	isEmpty	isEmpty	$O(1)$
size	size	size	$O(1)$
front	-	front	$O(1)$
back	peek	-	$O(1)$

Coincide perfectamente con lo visto en la tabla teórica, lo cual muestra que los métodos se comportaron tal como se esperaba. En algún momento del desarrollo hubo medidas que eran lineales sin razón. La causa era porque había un bucle para llenar las listas de antemano ¡siendo medido dentro de los resultados!. Ya está corregido y ahora los resultados tienen sentido.

Comparación de métodos equivalentes

Solo se compararán los métodos que estén en todas las estructuras de datos.

LinkedList.pushBack vs. Stack.push

Ambos añaden un elemento al final. En la lista enlazada es $O(1)$ gracias al puntero de cola (quien no lo tiene es `LinkedListNoTail`). En la pila es igualmente $O(1)$ porque no implica traslado de valores en el arreglo que lo implementa.

Sin embargo este método no es adecuado para implementar pilas con listas enlazadas como se explicará más adelante.

LinkedListNoTail.popFront vs. Queue.dequeue

En todas las listas enlazadas `popFront` es $O(1)$, pero como la cola implementa arreglo no circular, eliminar del principio en `dequeue` exige trasladar los elementos terminando degradando el rendimiento a $O(n)$.

La solución es un arreglo dinámico circular, pero la implementación actual solo es dinámico.

DoublyLinkedList.popBack vs. Stack.pop

El puntero a cola y el doble recorrido logran que eliminar el último elemento sea $O(1)$, mientras que en la pila es tan sencillo como retirar el último elemento porque tampoco hay traslados.

En una lista enlazada simple requiere $O(n)$ elementos retirar el final porque debemos buscar el penúltimo elemento. Por ello no eran eficientes para pilas. Usar la cabeza o una lista enlazada doble son mejores opciones.

DoublyLinkedList.erase vs. Stack.delete

Esta operación no es estándar de las pilas.

En una lista enlazada sobre la referencia al nodo a eliminar reduce la complejidad de eliminar a $O(1)$, aunque buscar la referencia sigue siendo $O(n)$. En las pilas sí es $O(n)$ porque implica trasladar los demás elementos de sitio.

isEmpty y size

Esto no exige alguna implementación a elegir: en todas es igual. Consisten en verificaciones sencillas o getters para variable.

Todas son $O(1)$.

¿Cuándo es mejor cada estructura?

Las listas enlazadas son utilizadas cuando la estabilidad y la abstracción son claves. Una lista doblemente enlazada, e incluso circular, pueden ser perfectas para almacenar datos de manera fragmentada y en tiempo más uniforme.

Las pilas y colas de arreglos dinámicos se usan cuando el rendimiento es más importante, y por supuesto, también con los casos de uso correspondientes. Un arreglo dinámico circular es la mejor opción para una cola, pero no fue implementado.

Sin embargo, las listas enlazadas pueden no aprovechar la memoria contigua del computador haciendo que el CPU busque a lo bruto los datos sin chance de estimar donde están (los llamados *cache misses*). Además no soportan acceso aleatorio (por índice), por lo que son menos flexibles en manejo.

Los arreglos dinámicos por su parte, pueden no tener garantizado su funcionamiento (porque exigen memoria contigua arbitrariamente grande) lo cual las puede hacer inestables. Además las reasignaciones generan picos en el procesamiento.

Casos de uso reales

Las listas enlazadas se pueden usar en contextos donde la POO ya está incluida (aunque no es obligatoria para tener listas enlazadas) o cuando la estabilidad es crítica. Esto incluye:

1. Gestores de memoria (`malloc`) e indexado
2. Planificadores en kernels
3. *Object pools* para reutilización, por ejemplo en videojuegos
4. Implementar pilas y colas (usos más adelante)

Los arreglos dinámicos suelen ser mucho más flexibles y completos, por lo que se usan en:

1. Algoritmos de ordenamiento y clasificación
2. Traslado y organización de datos
3. Soportan estructuras tales como árboles, pilas, colas, etc.
4. Matrices y tensores

Finalmente las pilas y las colas permiten esquemas de priorización y orden sencillos (LIFO y FIFO) que se usan en:

1. Compiladores, intérpretes, analizadores de sintaxis y validación
2. Deshacer y rehacer
3. Gestión de eventos y entradas (*input buffering* y *stacks* de navegación)
4. Buffers de todos los tipos (audio, video, UI, frames, ...)

Conclusiones

Hemos hecho un análisis teórico y empírico sobre el funcionamiento de las listas enlazadas en todas sus formas, las pilas y las colas y hecho supuestos en base a su funcionamiento y a análisis simples.

La experimentación demostró que la complejidad de las estructuras de datos coincidió con lo esperado y demostrado a lo largo de este informe, gracias a la implementación y medición hechos en Java.

Estos resultados son esenciales en el manejo real de estructuras de datos, las cuales se usan en todos los casos de uso posibles dentro del software real. Su comprensión nos permitirá elegir la mejor estructura para cada contexto.